

Description:

SpecSentinel is a professional-grade, single-file specification validation studio built in Python and Streamlit. The application provides an end-to-end, interactive pipeline that evaluates whether a candidate specification truly and completely captures the intended behaviour of a software system.

The platform integrates four academically grounded validation methods — automated behavioural testing, mutation kill analysis, bidirectional cross-checking, and lightweight formal property verification — into a cohesive, professional UI that produces actionable grades, detailed diagnostics, and exportable JSON reports.

SpecSentinel requires no external dependencies beyond Streamlit and runs instantly with a single terminal command. It ships with a fully worked demonstration example (a Safe Integer Stack specification) reviewers to experience the full validation pipeline in under two minutes.

Project Goals:

- Provide a unified, interactive platform covering all four validation methods named in the hackathon track.
- Generate actionable, quantitative quality metrics (scores, kill ratios, consistency percentages, letter grades).
- Enable zero-friction deployment, one file, one command, no configuration.
- Deliver a professional UI suitable for demonstration to international judges.
- Include a worked example to make the system immediately understandable.

GitHub Link: <https://github.com/mudit108-code/SpecSentinel>

Live Link: <https://specsentinel.streamlit.app/>

(NOTE: If you see a Blue Box, just click on it to access the app.)

Example Usage:

LEFT PANEL — Specification

Specification: Secure Bank Account

Preconditions

- balance ≥ 0
- account_id is a non-empty string

- `daily_limit > 0`
- `daily_limit <= 50000`
- `withdrawal_count >= 0`

Postconditions

- `deposit(amount)`: IF `amount > 0` THEN `balance == old_balance + amount`
- `deposit(amount)`: IF `amount <= 0` THEN RAISES `ValueError`
- `withdraw(amount)`: IF `amount > 0` AND `amount <= balance` AND `amount <= daily_limit` THEN `balance == old_balance - amount`
- `withdraw(amount)`: IF `amount > balance` THEN RAISES `InsufficientFundsError`
- `withdraw(amount)`: IF `amount > daily_limit` THEN RAISES `DailyLimitExceededError`
- `withdraw(amount)`: IF `amount <= 0` THEN RAISES `ValueError`
- `get_balance()`: returns current balance AND balance unchanged
- `reset_daily()`: `withdrawal_count == 0`

Invariants

- INV_1: `balance >= 0` at all times
- INV_2: All amounts are positive integers
- INV_3: `withdrawal_count >= 0`
- INV_4: `account_id` is immutable after creation

Safety Properties

- SAFE_1: balance never goes negative under any operation
- SAFE_2: No unauthorized access to `account_id`
- SAFE_3: Transaction atomicity — partial updates never occur

RIGHT PANEL — Program

class BankAccount:

```
def __init__(self, account_id: str, daily_limit: int = 10000):
    if not account_id or not isinstance(account_id, str):
        raise ValueError("account_id must be a non-empty string")
    if daily_limit <= 0 or daily_limit > 50000:
        raise ValueError("daily_limit must be between 1 and 50000")
    self._account_id = account_id
    self._balance = 0
    self._daily_limit = daily_limit
    self._withdrawal_count = 0

def deposit(self, amount: int):
    if not isinstance(amount, int) or amount <= 0:
        raise ValueError("Deposit amount must be a positive integer")
    self._balance += amount

def withdraw(self, amount: int):
    if not isinstance(amount, int) or amount <= 0:
        raise ValueError("Withdrawal amount must be a positive integer")
    if amount > self._balance:
        raise InsufficientFundsError("Insufficient funds")
    if amount > self._daily_limit:
        raise DailyLimitExceededError("Daily limit exceeded")
    self._balance -= amount
    self._withdrawal_count += 1
```

```
def get_balance(self) -> int:  
    return self._balance
```

```
def reset_daily(self):  
    self._withdrawal_count = 0
```

```
@property
```

```
def account_id(self) -> str:  
    return self._account_id
```